What Does the First Level of a Residual Text Tokenizer Know?

Learned-Query Pooling Puts the Gist in Level 1

Timoth Dev A
Independent researcher
timontunes@gmail.com

Abstract

Coarse-to-fine text generators such as Next-Concept Modeling (NCM) rely on a residual vector-quantized tokenizer whose first levels should carry the gist of a sentence. NCM builds each coarse level by pooling fixed windows of positions. We replace this with learned-query pooling (CruxTok, T1) and train on XSum summaries. Learned pooling makes level 1 recover much more of a sentence: against an NCM baseline with the same parameter count, T1's level-1 reconstructions reach LLM-judge win rates of 0.74 and 0.77 (two seeds, ties counted half; the judge picks T1 outright in 57% and 63% of items and the baseline in 9% and 8%), and embedding similarity to the original rises from 0.34 to 0.45. T1 also reconstructs 13–15 points more tokens from the first three levels. The pooling attention moves away from its starting prior and depends on the input. The parameter-matched baseline, which spends its extra parameters on encoder and decoder depth, gives codes whose similarity tracks STS-B better (Spearman 0.28 at level 3, against 0.02–0.03 for T1). Decodable meaning and meaning-organized codes are separate properties, and pooling buys the first but not the second. We also report the stabilizer needed to train learned pooling at all, QK-normalization against attention-logit growth. Code, configurations, evaluation outputs and trained tokenizers are public.

1 Introduction

A residual vector quantizer (RVQ) encodes an input as a stack of discrete codes, each level coding what the previous levels missed [13, 15]. NCM [6] applies this to text: a tokenizer maps a window of tokens to a few coarse codes followed by progressively finer ones, and a generator writes the coarse codes first. The approach only pays off if the first levels hold the gist of the sentence. In NCM each coarse level is formed by averaging fixed windows of positions, so level 1 is shaped by where tokens sit rather than by what they say.

We ask whether the first levels can be made to carry more meaning. We replace fixed pooling with learned-query pooling, where each code at a level attends over the encoded tokens (§3). We compare against NCM retrained under the same budget, and against an NCM with the same parameter count as ours, to separate the effect of pooling from that of model size.

Our findings:

- **Learned pooling puts more decodable meaning into level 1** (§5.2). Against both NCM baselines, on two seeds, by an embedding measure and by a validated LLM judge. The gain is largest at level 1 and gone by level 2.
- **The queries choose tokens.** Pooling attention moves away from its starting prior and differs between inputs.
- **It does not organize the code space by meaning** (§5.3). A parameter-matched NCM with a deeper encoder gives codes whose similarity tracks STS-B far better than learned pooling.

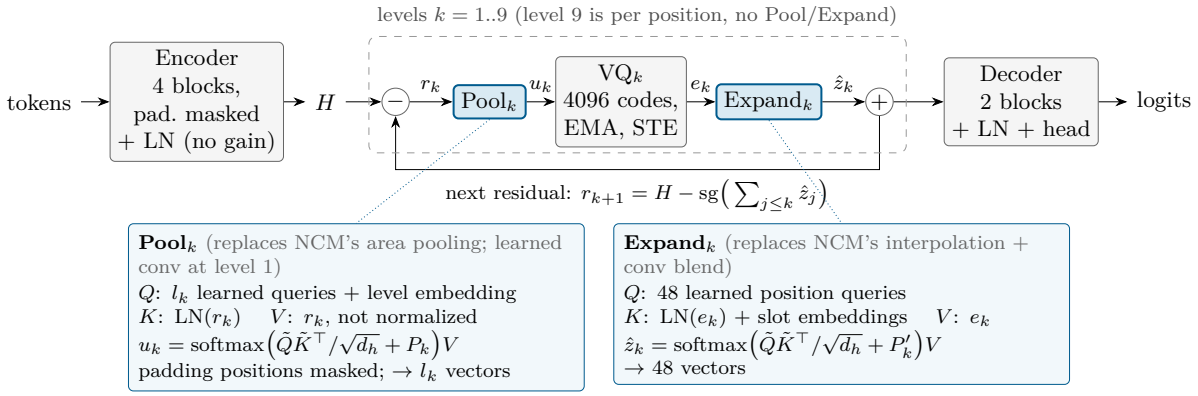


Figure 1: The T1 (CruxTok) tokenizer. Blue boxes replace NCM's per-level pooling and expansion; the residual loop, codebooks and nested dropout follow NCM. Section 3 lists the other differences from the baselines (encoder and decoder depth, padding masking, loss terms, code restarts). In both attention blocks (linear projections omitted), $\tilde{Q}, \tilde{K}$ are RMS-normalized per head (QK-norm, §6) and P_k, P'_k are learnable priors initialized to NCM's windows (Figure 2). Pooling values are the raw residual, so u_k keeps the residual's scale, which shrinks with level. LN: LayerNorm; sg: stop-gradient. During training the decoder receives the sum up to a randomly truncated level (nested dropout).

- **QK-normalization is required** (§6) to train learned pooling and expansion in bf16.

The work is small in scale: one dataset, a 48-token window, and a training budget about 16 times below NCM's.

2 Background: NCM's residual tokenizer

For XSum, NCM's released configuration uses nine levels with $[4, 6, 10, 14, 20, 26, 32, 40, 48]$ codes per level over a 48-token window, so the last level has one code per token. Each level k quantizes a pooled residual into l_k vectors from a 4096-entry codebook, expands it back to full length with an operator R_k, and subtracts it from the residual. Nested dropout [7, 12] truncates the stack at a random level during training so that every prefix decodes. NCM pools by area-downsampling fixed windows (with a learned convolutional cascade at level 1) and expands by interpolation followed by a convolutional blend. We write k for the cumulative use of levels $1..k$: k=1 is 4 codes, k=3 is 20, and "full" is all nine levels.

3 Method

Learned-query pooling (T1). We call this tokenizer CruxTok, after the goal that its first level keep the crux of a sentence; the code and checkpoints use that name. Figure 1 shows the whole tokenizer. A transformer encoder (4 blocks, d=384, 6 heads) contextualizes the token embeddings, and its output passes through a LayerNorm without learned gain. At level k, l_k learned queries cross-attend over the current residual, as in PMA, Perceiver and Q-Former [5, 8, 9]. Plain learned queries collapsed: every slot at a level pooled the same vector. We therefore add a learnable attention prior to the logits, initialized to a Gaussian centred on each of NCM's windows, so that slots start apart and can move (Figure 2a). For the reported runs the prior's sharpness is 1.0, chosen on held-out reconstruction in short pilot runs. At this sharpness the prior is wider than NCM's windows: at level 1 it spans 21.3 effective tokens against NCM's 12, with on average 75% of its mass inside its own window. Expansion is the mirror image: L learned position queries attend over the l_k quantized vectors (with learned slot embeddings

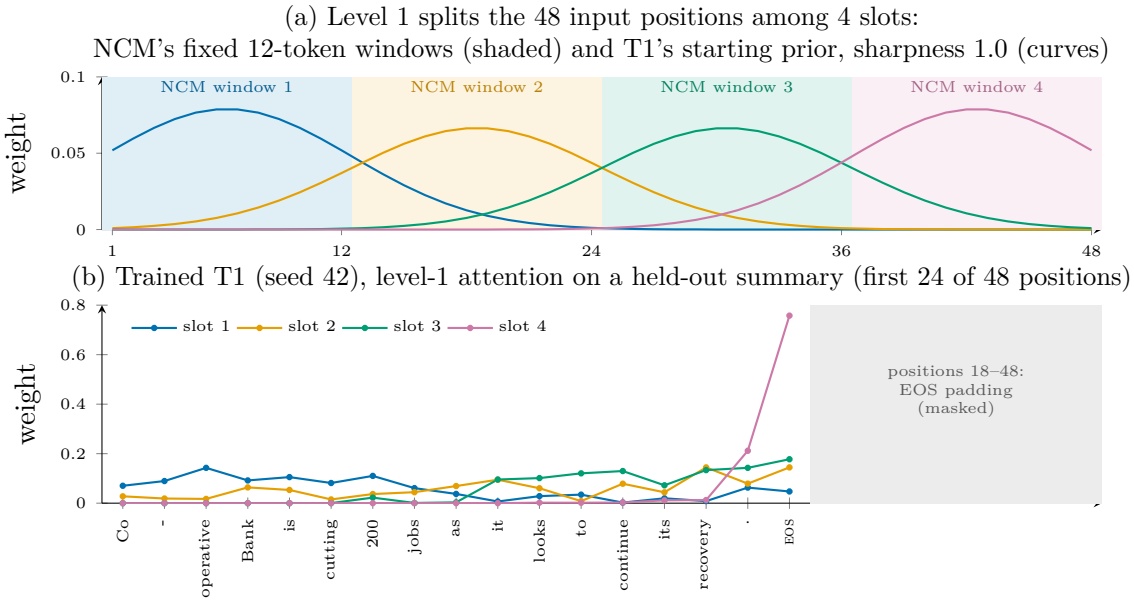


Figure 2: Learned-query pooling at level 1. (a) NCM pools four fixed 12-token windows. T1 has four learned queries whose attention starts from a Gaussian prior centred on those windows (sharpness 1.0), wider than the windows themselves. (b) After training, on a held-out XSum summary drawn at random (seed 0) that fills 17 of the 48 positions: NCM's windows 3 and 4 would pool only EOS padding, while all four T1 slots attend to the text and none to padding. T1's slots are not confined to their windows: the prior is a soft bias, and padding is masked, so slots 3 and 4 spread onto the nearest text. Slot 4 puts most of its mass (0.76) on the first EOS.

added to the keys) under their own learnable window prior. In both, keys are layer-normalized while values are not, so pooled vectors keep the residual's scale; pooling queries also carry a learned per-level embedding. Padding positions (NCM fills the window with EOS after the first EOS) are masked from the encoder's self-attention and from pooling attention; NCM masks no positions. The last level is quantized per position, as in NCM, and a 2-block decoder maps the summed reconstruction to token logits. We use a straight-through estimator [1] at every level, with each residual detached from previous levels' quantized output.

Objective. The loss is token cross-entropy over the whole window plus, averaged over levels, NCM's terms $\beta\|z - \mathrm{sg}(\hat{z}_k)\|^2 + \|\hat{z}_k - \mathrm{sg}(z)\|^2$ and a per-level commitment term $\beta\|u_k - \mathrm{sg}(e_k)\|^2$ that NCM does not use, with $\beta = 0.25$. Here u_k is the pooled residual, e_k its codebook vector and $\hat{z}_k$ the reconstruction from levels $1..k$. T1's codebooks also restart codes whose usage falls below 1% of uniform; NCM's do not.

Stabilizer. Pooling and expansion attention use QK-normalization; Section 6 explains why.

4 Experimental setup

Training. All tokenizers are trained on XSum training summaries [10] for 30,000 updates at batch 256, with AdamW (learning rate 5×10^{-4} cosine-decayed to 5×10^{-5}, 1,000 warmup steps, β_2=0.95, weight decay 0.01, clipping at 1.0), EMA codebooks and nested dropout with probability 0.5. T1 accumulates two micro-batches of 128, so its EMA codebooks update twice per step and the nested-dropout level is drawn per micro-batch; T0-cap uses one batch of 256. This is about 16 times fewer sequences than NCM's tokenizer budget, so we compare only

Run	Description	Params	Seeds
T0-cap	NCM architecture and schedule, same budget	60.6M	42 (fp16), 2 (bf16)
T0-cap, matched	NCM with 5 encoder and 3 decoder blocks	73.1M	42 (bf16)
T1 (CruxTok)	learned-query pooling	73.2M	42, 2 (bf16)

Table 1: Runs. Parameter counts include codebooks, following NCM. The fp16 and bf16 T0-cap runs do not differ significantly on PAWS or STS-B at any level.

Run	k=1	k=2	k=3	Full
T0-cap (s42 / s2)	23.5 / 23.0	36.3 / 35.7	47.1 / 46.5	95.8 / 95.7
T0-cap, matched	19.3	32.3	45.6	97.8
T1 (s42 / s2)	23.5 / 24.4	41.0 / 40.4	**60.1 / 60.7**	**98.8 / 98.8**

Table 2: Token accuracy (%) on the text tokens, including the first EOS, of the first 2,000 XSum validation summaries, decoding from levels 1..k.

against NCM retrained under the same budget (T0-cap), never against the released checkpoint.

Code similarity. The similarity of two sentences at level k is the cosine between the means of their level 1..k codebook vectors. We report PAWS AUC [labeled_final test, 8,000 pairs; 16] and STS-B Spearman [test, 1,379 pairs; 2], both used for evaluation only. Differences use a paired bootstrap over pairs (2,000 resamples, 95% CI). Floors are random scores, word Jaccard and TF-IDF cosine; the ceiling is all-mpnet-base-v2 [11]. A secondary measure averages the level-k reconstruction over non-padding positions, since on STS-B the primary measure correlates with sentence length (ρ with the shorter sentence's length between -0.20 and -0.71 across tokenizers at k=1–3, and -0.63 to -0.71 for T1) when most of the window is EOS filler.

Decodable meaning (protocol fixed before results). We decode 1,000 random XSum test summaries from levels 1..k only. Measure A is the cosine between Qwen3-Embedding-8B embeddings of the original and its reconstruction, with a paired bootstrap over items. Measure B is a pairwise LLM judge [17]: the judge sees the original and two reconstructions at the same k and answers which keeps more of who, what, where and the outcome, ignoring fluency. Each item is judged in both orders, and a side wins only if both orders agree; win rate is (wins + ties/2)/n with a bootstrap CI, where ties include order disagreements and n counts the items with usable answers in both orders (991–999 of 1,000). Four candidate judges were scored on STS-B before use; we use GLM-5.3 with low reasoning effort (Spearman 0.897 with human scores, order consistency 0.94). One candidate, Kimi-K2.5, was excluded because 38% of its answers were empty.

5 Results

5.1 Reconstruction

Table 2 shows that T1 reconstructs better than both baselines at every depth beyond level 1, by 13–15 points at k=3. Extra encoder and decoder depth in the matched baseline helps at full depth (95.8 to 97.8) but not at the coarse levels.

5.2 Learned pooling puts more decodable meaning into level 1

Figure 3 and Table 3 show the main positive result. With four codes, T1 reconstructions are about 0.1 closer to the original in embedding space than either baseline, on both seeds. The judge agrees: T1's level-1 win rate is 0.70 against T0-cap, and 0.74 and 0.77 against the matched

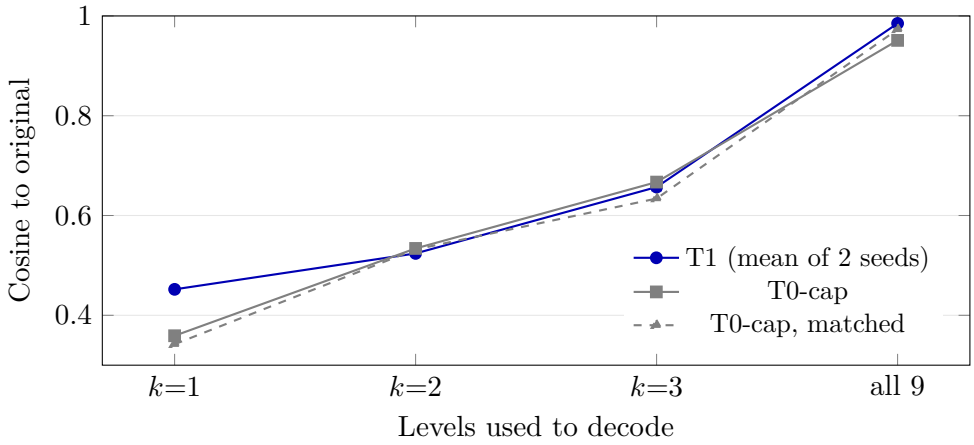


Figure 3: Measure A: embedding cosine between 1,000 XSum test summaries and their reconstructions from levels 1..k. Error bars span the two T1 seeds.

	k=1	k=2	k=3	Full
Measure A: embedding cosine difference				
T1 s42 − T0-cap	**+0.090**	−0.006	−0.002	**+0.034**
T1 s2 − T0-cap	**+0.097**	**−0.013**	**−0.017**	**+0.033**
T1 s42 − T0-cap, matched	**+0.107**	−0.004	**+0.031**	**+0.012**
T1 s2 − T0-cap, matched	**+0.113**	**−0.011**	**+0.016**	**+0.011**
Measure B: LLM-judge win rate				
T1 s42 vs T0-cap	**0.700** [0.678, 0.722]	0.489 [0.463, 0.516]	**0.541** [0.515, 0.568]	–
T1 s42 vs T0-cap, matched	**0.740** [0.720, 0.760]	n/s	n/s	–
T1 s2 vs T0-cap, matched	**0.772** [0.753, 0.792]	n/s	n/s	–

Table 3: Decodable meaning on 1,000 XSum test summaries. T0-cap is seed 42. Bold: 95% CI excludes 0 (measure A) or 0.5 (measure B). n/s: not scored (see §7). Measure-A CI half-widths are at most 0.009.

baseline. Counting outright preferences only, the judge picks T1 in 52% of items against T0-cap (12% for T0-cap), and in 57% and 63% against the matched baseline (9% and 8%); the rest are ties or order disagreements. The matched baseline is the weakest of the three at level 1, so the gain is not explained by parameter count. By k=2 the runs are within about 0.01 of each other.

Table 4 shows reconstructions drawn at random. At level 1, T1 often preserves the type of event (an injured footballer, a crash at a racing championship) while getting names, places and numbers wrong, though on the boxing item it keeps little; the matched baseline's level 1 is mostly unrelated text, except on formulaic sentences such as the murder arrest.

Do the queries choose tokens? If pooling attention stayed on its starting prior, T1 would be close to NCM with extra parameters. On 256 held-out windows, pooling places zero mass on padding (44% of positions), so we measure movement over text positions only, renormalizing both the attention and the starting prior. At levels 1–3, the attention on text differs from the starting prior by 37–53% (total-variation distance per input, so this includes the between-input variation), the attention differs between inputs by 22–41% (total-variation distance from the mean map), and it is narrower than the starting prior (14.6 vs 21.3 effective tokens at level 1, the prior counted over all 48 positions), though still wider than NCM's 12-token window. Both seeds agree within 0.04. The learned prior itself moves by only 0.05 at level 1, so the movement there comes from the queries. The starting prior is not the attention at initialization: with QK-normalization the query–key logits are already of order 1 at step 0, and we did not measure attention there. Figure 2b shows one example. Movement rules out fixed pooling; it does not

Original	T1, k=1	T0-cap matched, k=1
West Ham striker Andy Carroll is facing up to six weeks out of action with a hamstring injury.	Newcastle midfielder Aaron Aaron has has he be surgery the out of the of after ankle injury.	Aales Minister Alex Johnson is preparing to his his. ahead of the.
Michael Dunlop set the fastest times for the Ulster Grand Prix Superbike and Superstock classes in Thursday's delayed practice sessions at Dundrod.	A lorry driver who crashed been the crash at the World Tour Championships Championships in the the on Saturday the the Grand Grand in in in London.	When's of has a a to record for the second in a the of the- his, in England, the too on the years.
A man has been arrested on suspicion of murder after a woman was found dead at a house in Worcestershire.	A man has been arrested on suspicion of murder after a woman woman was a in in in in indearkshire.	A man has been arrested on suspicion of murder after a woman- a murder at a house in arexhamhire.
A former Welsh amateur boxing champion said she is turning professional as she does not have confidence in the governing body.	A former former assistant boss has has been been been described as as as of the the the the the election.	A man of David Cameron has said he will to the a" of a to hospital in the UK.

Table 4: Level-1 reconstructions. Four of six items drawn uniformly at random (seed 0) from the 1,000 evaluated; the other two, and k=3 outputs, are in the repository.

	PAWS AUC			STS-B Spearman		
	k=1	k=2	k=3	k=1	k=2	k=3
T0-cap (s42)	0.546	0.549	0.551	−0.058	−0.029	0.067
T0-cap, matched	0.547	**0.571**	**0.579**	−0.009	**0.102**	**0.282**
T1 (s42 / s2)	0.551 / **0.561**	0.563 / 0.562	0.566 / 0.563	−0.033 / **0.007**	−0.019 / −0.007	0.022 / 0.027
Floors: random / Jaccard / TF-IDF	0.505 / 0.495 / 0.484			0.000 / 0.565 / 0.693		
Ceiling: all-mpnet-base-v2	0.652			0.834		

Table 5: Code similarity, primary measure. Bold: best tokenizer in the column.

show that attention moves toward meaning.

5.3 Pooling does not organize the code space by meaning

Decoding a sentence from its own codes and giving similar sentences similar codes are different properties. Table 5 shows that the parameter-matched baseline is best at the second. Against it, T1 is behind at k=2–3 on both tasks and both seeds. The CIs exclude 0 everywhere except PAWS at k=2 (−0.008 [−0.019, +0.003] for both seeds). On STS-B at k=3 the gap is −0.259 [−0.314, −0.208] for seed 42 and −0.255 [−0.311, −0.206] for seed 2. At k=1, T1 is ahead on PAWS (+0.004 and +0.014; significant for seed 2 only), and STS-B is mixed (−0.024 and +0.016, neither significant). The padding-aware secondary measure agrees (STS-B at k=3: 0.257 for the matched baseline against 0.147 and 0.116 for T1). In absolute terms all tokenizers are weak sentence-similarity models: word Jaccard alone reaches 0.565 on STS-B, and on PAWS every tokenizer scores 0.55–0.58 on the primary measure against 0.65 for the sentence-encoder ceiling.

6 Training stability

Attention-logit growth. T1 diverged in bf16 under per-level STE. The expansion attention's query and key weights grew about 50-fold and its logits reached about 10^7, the failure described by Dehghani et al. [3] and Wortsman et al. [14]. RMS-normalizing queries and keys per head [4], with the scale fixed so that the query–key term of the logits is bounded by $\sqrt{d_{\text{head}}} = 8$ (the learned prior is added on top), removed the divergence.

A fallback that fails. Applying STE only at the coarse levels, and NCM's global path below, left the deeper learned pools without a reconstruction gradient; their outputs collapsed to an RMS of 0.005. With learned pooling, every level needs its own gradient path.

7 Limitations

The study uses one dataset of single-sentence summaries, a 48-token window and a reduced training budget. No generator was trained, so whether a more decodable level 1 improves coarse-to-fine generation is untested; that is the main open question. The judge was scored against the matched baseline only at k=1: at k=3 two runs failed our 5% missing-answer guard because the judge sometimes exhausted its token budget reasoning, and a retry ran out of API credit; we report these as not scored. The matched baseline has one seed. Measure A partly rewards fluent reconstructions, which is why we pair it with the judge, which is told to ignore fluency.

8 Conclusion

Learned-query pooling makes the first four codes of a residual text tokenizer recover substantially more of a sentence than NCM's fixed pooling, including against an NCM with the same parameter count. It does not make similar sentences receive similar codes; a deeper NCM encoder does that better. For coarse-to-fine generation, the property that matters is likely decodability of the coarse levels, and that is the property pooling improves.

Reproducibility. Code, configurations, evaluation outputs and the level-1 reconstructions used by the judge are at `https://github.com/Zilch123/CruxTok`. Trained tokenizers are at `https://huggingface.co/TimothDevA/cruxtok`.

References

[1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.

[2] Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of SemEval*, 2017.

[3] Mostafa Dehghani et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, 2023.

[4] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of EMNLP*, 2020.

[5] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning*, 2021.

[6] Hermann Kumbong, Ishan S. Khare, Iddah Mlauzi, Andrew Woen, Charles Ding, E. Kelly Buchanan, Simran Arora, Azalia Mirhoseini, and Christopher Ré. Next-concept modeling: Fast, parallel coarse-to-fine text generation, 2026.

[7] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and

Ali Farhadi. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, 2022.

[8] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning*, 2019.

[9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, 2023.

[10] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In *Proceedings of EMNLP*, 2018.

[11] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[12] Oren Rippel, Michael Gelbart, and Ryan Adams. Learning ordered representations with nested dropout. In *International Conference on Machine Learning*, 2014.

[13] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.

[14] Mitchell Wortsman et al. Small-scale proxies for large-scale transformer training instabilities. In *International Conference on Learning Representations*, 2024.

[15] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021.

[16] Yuan Zhang, Jason Baldridge, and Luheng He. PAWS: Paraphrase adversaries from word scrambling. In *Proceedings of NAACL*, 2019.

[17] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.